

Beginner Tutorial: Your First VM Migration

Duration: 30-45 minutes **Difficulty:** Beginner **Prerequisites:** Basic command-line knowledge, VM disk image file

What You'll Learn

By the end of this tutorial, you will: - Install Hyper2KVM and dependencies - Perform your first VM migration - Understand automatic fixes applied during migration - Validate the migrated VM - Import the VM into libvirt/KVM

Prerequisites

System Requirements

- **OS:** Linux (Ubuntu 20.04+, Fedora 35+, RHEL 8+, or similar)
- **Python:** 3.10 or later
- **Disk Space:** 3x the size of your largest VM
- **RAM:** 4GB minimum, 8GB recommended
- **Privileges:** sudo/root access for mounting disk images

Required Software

- Python 3.10+
 - qemu-img
 - libvirt (optional, for running VMs)
-

Step 1: Installation

Install via pip (Recommended)

```
# Install Hyper2KVM
pip install hyper2kvm

# Verify installation

> **Note**: After installation, you have two command names:
> - `h2kvmctl` (primary, kubectl-style, recommended)
> - `hyper2kvm` (legacy, backwards compatible)
>
> Both commands are identical. This tutorial uses `h2kvmctl`.

h2kvmctl --version
```

Expected output:

```
hyper2kvm version 1.0.0
```

Install System Dependencies

Ubuntu/Debian:

```
sudo apt-get update
sudo apt-get install -y \
    qemu-utils \
    libvirt-clients \
    libvirt-daemon-system
```

Fedora/RHEL:

```
sudo dnf install -y \
    qemu-img \
    libvirt-client \
    libvirt-daemon
```

Verify Installation

```
# Check guestfs
guestfish --version

# Check qemu-img
qemu-img --version

# Check virsh (optional)
virsh --version
```

Step 2: Prepare Your Source VM

For this tutorial, we'll migrate a Windows Server 2019 VM from Hyper-V to KVM.

Locate Your VM Disk

Hyper-V VMs are typically in: - `/var/lib/hyperv/` (Linux Hyper-V) - `C:\Users\Public\Documents\Hyper-V\Virtual Hard Disks\` (Windows Hyper-V)

VMware VMs are typically in: - `/vmfs/volumes/datastore1/` (ESXi) - User's Documents folder (VMware Workstation)

Example:

```
# Copy VM from Windows host to Linux migration server
scp user@hyperv-host:/path/to/windows-server.vhdx /vms/source/

# Or mount network share
mount -t cifs //hyperv-host/vms /mnt/hyperv -o username=admin
cp /mnt/hyperv/windows-server.vhdx /vms/source/
```

Check Disk Format

```
qemu-img info /vms/source/windows-server.vhdx
```

Expected output:

```
image: /vms/source/windows-server.vhdx
file format: vhdx
virtual size: 127 GiB (136365211648 bytes)
disk size: 45.2 GiB
cluster_size: 1048576
```

Step 3: Your First Migration

Basic Migration Command (Using YAML Config - Recommended)

```
# migration.yaml
cmd: local
vmdk: /vms/source/windows-server.vhdx
output_dir: /vms/migrated
to_output: windows-server.qcow2
out_format: qcow2
fstab_mode: stabilize-all
regen_initramfs: true
windows: true
virtio_win_iso: /usr/share/virtio-win/virtio-win.iso
compress: true
verbose: 1
```

```
h2kvmctl --config migration.yaml
```

Alternative: CLI Flags

```
h2kvmctl --cmd local \
  --vmdk /vms/source/windows-server.vhdx \
  --output-dir /vms/migrated \
  --to-output windows-server.qcow2 \
  --out-format qcow2 \
  --fstab-mode stabilize-all \
  --regen-initramfs \
  --windows \
  --virtio-win-iso /usr/share/virtio-win/virtio-win.iso \
  --compress \
  --verbose
```

Understanding the Command

Option	Description
<code>--cmd local</code>	Process local disk file migration
<code>--vmdk</code>	Source VM disk image (VMDK, VHDX, VHD, etc.)
<code>--output-dir</code>	Output directory for converted VM
<code>--to-output</code>	Output filename
<code>--out-format qcow2</code>	Target format (qcow2, raw, vmdk, etc.)
<code>--fstab-mode stabilize-all</code>	Stabilize fstab with UUIDs
<code>--regen-initramfs</code>	Regenerate initramfs with virtio drivers
<code>--windows</code>	Enable Windows guest mode
<code>--virtio-win-iso</code>	Path to VirtIO driver ISO for Windows
<code>--compress</code>	Compress output (for qcow2)
<code>--verbose</code>	Show detailed progress

Migration Progress

You'll see output like:

```
[INFO] Starting migration...  
[INFO] Source: /vms/source/windows-server.vhdx (VHDX, 127 GiB)  
[INFO] Target: /vms/migrated/windows-server.qcow2 (QCOW2)  
  
[1/7] Converting disk format...  
██████████████████████████████████████ 100% (45.2 GiB)  
  
[2/7] Launching VMCraft...  
✓ NBD device connected: /dev/nbd0  
✓ Partitions detected: 3  
  
[3/7] Mounting filesystems...  
✓ /dev/nbd0p1: EFI System Partition (FAT32)  
✓ /dev/nbd0p2: C:\ (NTFS)  
✓ /dev/nbd0p3: Recovery (NTFS)
```

```
[4/7] Detecting OS...
  ✓ OS: Windows Server 2019 Standard
  ✓ Edition: Datacenter
  ✓ Build: 17763

[5/7] Applying fixes...
  ✓ Bootloader: Configured for KVM
  ✓ Network: VirtIO drivers installed
  ✓ Storage: VirtIO SCSI drivers installed
  ✓ fstab: Not applicable (Windows)

[6/7] Cleaning up...
  ✓ Filesystems unmounted
  ✓ NBD device disconnected

[7/7] Migration complete!
  ✓ Target: /vms/migrated/windows-server.qcow2
  ✓ Size: 45.2 GiB compressed to 38.1 GiB
  ✓ Duration: 8m 23s
```

Step 4: Understanding Automatic Fixes

Hyper2KVM automatically applies these fixes during migration:

Bootloader Fix

Problem: Hyper-V/VMware bootloader won't work on KVM **Fix:** Reconfigures GRUB (Linux) or BCD (Windows) for KVM hardware

Example (Linux): - Updates `/etc/default/grub` with KVM-compatible settings - Regenerates GRUB configuration - Installs GRUB to boot partition

Example (Windows): - Updates Boot Configuration Data (BCD) - Configures for UEFI or BIOS boot - Ensures boot device is correct

Network Fix

Problem: Hyper-V/VMware network drivers incompatible with KVM **Fix:** Installs VirtIO network drivers, configures network interfaces

Example (Linux): - Detects old network interface names (eth0, ens160) - Creates new configuration for virtio interfaces - Preserves IP addresses and network settings

Example (Windows): - Installs VirtIO network drivers - Configures network adapter for KVM - Preserves static IP configuration

Storage Driver Fix

Problem: Hyper-V/VMware storage drivers won't work on KVM **Fix:** Installs VirtIO SCSI drivers for disk access

Example (Linux): - Adds `virtio_scsi` to initramfs - Regenerates initramfs with new drivers

Example (Windows): - Injects VirtIO SCSI drivers - Updates Windows registry for new storage - Ensures boot disk is accessible

fstab Stabilization (Linux only)

Problem: Device names change (e.g., `/dev/sda` → `/dev/vda`) **Fix:** Converts device names to UUIDs for stability

Example:

```
# Before migration (unstable)
/dev/sda1 /boot ext4 defaults 0 1
/dev/sda2 /      ext4 defaults 0 0

# After migration (stable)
UUID=abc123... /boot ext4 defaults 0 1
UUID=def456... /      ext4 defaults 0 0
```

Step 5: Validate the Migration

Post-Migration Verification

After migration, verify the converted disk image:

```
# Check the output file
qemu-img info /vms/migrated/windows-server.qcow2

# Boot test (optional - requires libvirt)
h2kvmctl --cmd local \
    --vmdk /vms/migrated/windows-server.qcow2 \
```

```
--libvirt-test

# Manual inspection
virt-inspector /vms/migrated/windows-server.qcow2
```

Migration Success Indicators

Look for these in the migration output:

```
✓ Disk conversion successful
✓ Bootloader configuration updated
✓ Network configuration stabilized
✓ fstab updated with UUIDs
✓ initramfs regenerated with virtio drivers
✓ Output saved to: /vms/migrated/windows-server.qcow2
```

Verify Output File

```
# Check the converted disk
qemu-img info /vms/migrated/windows-server.qcow2

# Expected output:
# image: /vms/migrated/windows-server.qcow2
# file format: qcow2
# virtual size: 127 GiB
# disk size: 38.1 GiB (compressed)
```

Step 6: Import to Libvirt

Create Libvirt XML Definition

```
# Generate libvirt XML
cat > /etc/libvirt/qemu/windows-server.xml <<'EOF'
<domain type='kvm'>
  <name>windows-server</name>
  <memory unit='GiB'>8</memory>
  <vcpu>4</vcpu>
  <os>
    <type arch='x86_64' machine='pc-q35-6.2'>hvm</type>
    <boot dev='hd' />
  </os>
</domain>
```



```

</os>
<features>
  <acpi/>
  <apic/>
  <hyperv>
    <relaxed state='on' />
    <vapic state='on' />
    <spinlocks state='on' retries='8191' />
  </hyperv>
</features>
<cpu mode='host-passthrough' />
<clock offset='localtime'>
  <timer name='hypervclock' present='yes' />
</clock>
<devices>
  <disk type='file' device='disk'>
    <driver name='qemu' type='qcow2' cache='writeback' />
    <source file='/vms/migrated/windows-server.qcow2' />
    <target dev='vda' bus='virtio' />
  </disk>
  <interface type='bridge'>
    <source bridge='br0' />
    <model type='virtio' />
  </interface>
  <console type='pty' />
  <graphics type='vnc' port='-1' autoport='yes' />
</devices>
</domain>
EOF

```

Define and Start VM

```

# Define VM in libvirt
sudo virsh define /etc/libvirt/qemu/windows-server.xml

# List VMs to verify
sudo virsh list --all

# Start VM
sudo virsh start windows-server

# Check status
sudo virsh dominfo windows-server

```

Expected output:

```
Id:          1
Name:        windows-server
UUID:        abc123-def456-...
OS Type:     hvm
State:       running
CPU(s):      4
Max memory:  8388608 KiB
Used memory: 8388608 KiB
```

Connect to VM

```
# Console access
sudo virsh console windows-server

# Or VNC (check port)
sudo virsh vncdisplay windows-server
# Output: :0 (means localhost:5900)

# Connect with VNC client
vncviewer localhost:5900
```

Step 7: Post-Migration Verification

Check VM is Booting

1. **Watch boot process:** Use VNC/console to monitor boot
2. **Wait for login prompt:** Should appear within 1-2 minutes
3. **Login:** Use your original credentials

Verify Network Connectivity

Linux VM:

```
# Check IP address
ip addr show

# Test internet connectivity
ping -c 4 8.8.8.8
```

```
# Test DNS
ping -c 4 google.com
```

Windows VM:

```
# Check IP address
ipconfig /all

# Test internet connectivity
ping 8.8.8.8

# Test DNS
ping google.com
```

Verify Services

Linux VM:

```
# Check critical services
systemctl status sshd
systemctl status NetworkManager

# Check all running services
systemctl list-units --type=service --state=running
```

Windows VM:

```
# Check critical services
Get-Service | Where-Object {$_.Status -eq "Running"}

# Check specific service
Get-Service WinRM
```

Common Issues and Solutions

Issue 1: VM Won't Boot

Symptom: VM starts but doesn't reach login prompt

Solution:

```
# Re-run migration with bootloader and initramfs fixes
h2kvmctl --cmd local \
  --vmdk /vms/source/windows-server.vhdx \
  --output-dir /vms/migrated \
  --to-output windows-server.qcow2 \
  --regen-initramfs \
  --verbose
```

Issue 2: No Network Connectivity

Symptom: VM boots but no network access

Solution:

```
# Re-run with fstab stabilization and initramfs rebuild
h2kvmctl --cmd local \
  --vmdk /vms/source/windows-server.vhdx \
  --output-dir /vms/migrated \
  --to-output windows-server.qcow2 \
  --fstab-mode stabilize-all \
  --regen-initramfs \
  --verbose
```

Issue 3: Disk Not Detected (Windows)

Symptom: “No boot device found” or “Disk not detected”

Solution: 1. Ensure VirtIO drivers are injected during migration 2. Check libvirt XML uses `bus='virtio'` 3. For Windows, inject VirtIO drivers:

```
# Re-run with Windows VirtIO driver injection
h2kvmctl --cmd local \
  --vmdk /vms/source/windows-server.vhdx \
  --output-dir /vms/migrated \
  --to-output windows-server.qcow2 \
  --windows \
  --virtio-win-iso /usr/share/virtio-win/virtio-win.iso \
  --compress \
  --verbose
```

Next Steps

Congratulations! You've completed your first VM migration. Here's what to explore next:

Beginner → Intermediate

- [Batch Migration](#): Migrate multiple VMs
- [Configuration Files](#): Use YAML configs for repeatable migrations
- [Automation](#): Script migrations for CI/CD

Explore Features

- [Live Migration](#): Migrate running VMs with <5s downtime
- [Validation Framework](#): Deep validation checks
- [Rollback](#): Recover from failed migrations

OS-Specific Guides

- [Windows Migration](#): Windows-specific tips
 - [Linux Migration](#): Linux-specific configuration
-

Summary Checklist

- Installed Hyper2KVM and dependencies
 - Prepared source VM disk image
 - Executed first migration with automatic fixes
 - Validated migrated VM
 - Imported VM to libvirt
 - Verified VM boots and network works
 - Understand common issues and solutions
-

Getting Help

- **Documentation Hub:** [docs/index.md](#)
- **Troubleshooting:** [guides/troubleshooting.md](#)

- **Migration Recipes:** [recipes/01-common-scenarios.md](https://github.com/ssahani/hyper2kvm/blob/master/recipes/01-common-scenarios.md)
- **GitHub Issues:** <https://github.com/ssahani/hyper2kvm/issues>

Time to completion: 30-45 minutes

Next Tutorial: [Intermediate Workflows](#)